코어와 에이전트가 주고받는 값

triPilot(`final_project_cs`) 코드를 직접 읽어 정리 · 2026-10-01 · 읽기만 했고 코드는 바꾸지 않았습니다. 경로는 모두 `final_project_cs/` 아래입니다.

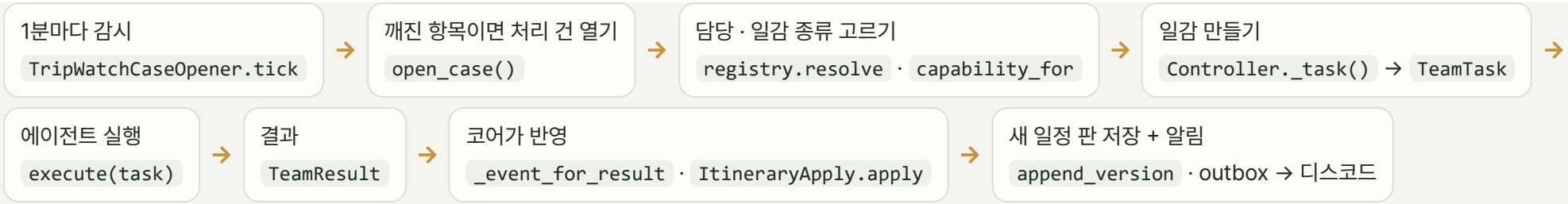
먼저 알아 두실 것 다섯 가지

1. **「코어 → 에이전트 → 코어」 경로를 실제로 타는 것은 감시가 연 처리 건뿐입니다.** 고객이 채팅으로 보낸 말(`POST /v1/trips/{trip_id}/messages`)은 처리 건(Case)을 만들고 분류는 하지만, 코어의 `Controller` 와 에이전트를 거치지 않고 `TripDesk` 가 직접 처리합니다. 이 문서의 2~11장은 감시 건 · `POST /v1/cases` 경로 기준입니다(고객 채팅 경로는 12장).
2. **「외부 데이터 6종을 3분마다 수집」은 결정(D-017, 초안)만 있고 코드에는 없습니다.** 지금 실제로 도는 것은 **1분마다**, 앞으로 **90분 안에 시작하는 일정 항목**을 하나씩 점검하는 방식입니다(`trip_watch_cases.py:39` , `scripts/ops/jobs.py:71-83`). 발표 자료의 「3분마다」는 「결정된 설계 · 미구현」으로 고쳐야 합니다.
3. **「사람 대기 없음」 결정(2026-09-28)**: 고객에게 「사람이 확인하도록 남겨 두었어요」 문구는 빠졌습니다. 하지만 코드의 상태 이름 `escalated` 는 남아 있고, 감시 건이 escalated가 되면 **고객에게 아무 알림도 가지 않습니다**(11장). 운영 화면 문구에도 「사람에게 옵니다」가 남아 있습니다(14장).
4. **숫자 대부분은 근거가 적혀 있지 않습니다.** 12,000자 · 90초 · 근거 40건 · 재시도 3번 등. 근거가 있는 숫자는 13장에 따로 모았습니다. 많은 숫자의 원 출처로 적힌 「구현계획서 v5」 파일은 워크스페이스에 없습니다.
5. **코드에서 문제를 여러 개 찾았습니다**(14장). 예: 이동 에이전트의 조회 도구 `read.route` · `read.transit` 은 구현되지 않아 늘 빈 값입니다.

목차 — 1 전체 흐름 · 2 입구 · 3 담당 고르기 · 4 일감(TeamTask) · 5 근거 묶음(ContextPack) · 6 에이전트 안 · 7 조회 도구 · 8 에이전트별 · 9 결과(TeamResult) · 10 코어가 반영 · 11 알림 · 12 고객 채팅 경로 · 13 숫자와 근거 · 14 확인된 문제 · 15 바로잡은 앞 설명

---

1. 전체 흐름 — 감시가 연 처리 건 기준



쉬운 말로: 감시가 「이 항목이 깨졌다」만 알아내 처리 건을 엽니다. 코어가 담당 에이전트와 일감 종류를 골라 일감 한 묶음을 넘깁니다. 에이전트는 허락된 도구로 다시 확인하고 「그대로 / 이렇게 바꾸자」를 결과로 돌려줍니다. 일정을 실제로 바꾸고 알림을 넣는 것은 코어입니다.

---

2. 입구 — 처리 건(Case)이 열리는 곳

처리 건(Case)이란: 고객 문의나 시스템이 찾은 사건 한 건입니다. DB 표 `customer_cases`에 한 줄로 저장되고, 상태(`status`)가 12종 중 하나로 바뀌어 갑니다. 상태가 바뀔 때마다 `case_events`에 기록이 쌓입니다(지우지 않음).

입구	정확한 이름	하는 일	코어·에이전트를 거치나
감시	`trip_watch_cases.py` · `TripWatchCaseOpener.tick`	앞으로 90분 안에 시작하는 항목을 점검해, 깨졌으면 `open_case(labels=…)`로 **시스템 처리 건**을 열고 `run_case`를 부릅니다. 무엇이 문제인지 이미 알아서 LLM 분류를 하지 않습니다(분류 값을 고정해 넣음). 같은 여행 · 항목 · 원인이면 같은 처리 건으로 모입니다(요청 id `watch:{trip}:{item}:{지문}`).	**예**
일반 처리 건 API	`POST /v1/cases` (`app/presentation/api/cases.py:120`)	처리 건을 열고, 응답을 먼저 보낸 뒤 뒤에서 `run_case`를 돌립니다(접수에서 실행까지 실측 20~51초라 기다리게 하지 않으려고).	예
추가 입력 API	`POST /v1/cases/{case_id}/messages`	기다리던 처리 건에 고객이 답하면 `Controller.resume` → `run_case` . 여행 에이전트는 이 「고객 답 기다리기」를 쓰지 않습니다(4장 `resume`).	예
여행 API (웹 · 개인 에이전트 앱)	`POST /v1/trips` · `/v1/trips/plan` · `/{trip_id}/messages` · `/reports` · `/rollback` ·	일정 등록 · 짜기 · 채팅 · 신고 · 되돌리기 · 다른 안 고르기. `TripStore` · `TripDesk`를 직접 씁니다.	**아니요**(12장)

	`/proposals/{proposal_id}/choose` · `/items/{item_id}/alternate`		

주기: 감시와 「멈춘 처리 건 되잡기」(`routing_sweeper`)는 같은 작업(`scripts.run_sweepers --once`)으로 **1분마다** 돕니다(`scripts/ops/jobs.py:71-83` , 윈도우 작업 스케줄러 `/SC MINUTE /MO 1`). 근거 있음 「주기는 안전이 아니라 복구 지연을 정한다」(`jobs.py:8`). 90분 앞을 보는 값은 근거 없음 .

되잡기(`routing_sweeper`): 담당이 정해지지 않은 채(`routing`) 10분(`routing_stuck_after_seconds: 600`) 넘게 멈춘 처리 건을 한 번에 최대 50건 다시 돌립니다. 600초는 근거 있음 「실행이 원래 수십 초 걸려 짧게 잡으면 정상 실행을 되잡아 같은 건을 둘이 돌린다」.

---

3. 담당 에이전트와 일감 종류 고르기

값 · 함수	무엇인가	어떻게 작용하나
`intent`	요청 종류. 여행은 다섯 가지: `itinerary_submit` (일정 제출) · `incident_report` (사건 신고) · `confirm_request` (확인 요청) · `adjust_reject` (조정 거부) · `other` (그 외) — `feedback.py:50-56`	요청 종류만으로는 어느 에이전트인지 정해지지 않습니다. 감시 건은 늘 `incident_report` 입니다.
`issue_code`	문제 코드. 앞부분이 담당 영역입니다. 예: `dining_hours` , `activity_weather_risk` , `mobility_missed_or_disrupted` — `feedback.py:63-82`	감시 건은 `activity_weather_risk` / `activity_other` / `dining_other` / `mobility_missed_or_disrupted` 를 고정해 넣습니다. `other` 는 담당이 없어 escalated로 갑니다.
`Controller._case_type()` → `case_type_of()`	처리 건의 종류를 정하는 함수(`routing.py:18-48`)	`issue_code` 의 `_` 앞부분을 씁니다(예: `dining_hours` → `dining`). 확인된 힌트가 있으면 힌트가 이깁니다.
`registry.resolve(case_type, intent)`	등록된 에이전트 목록(`config/project.yaml` 의 `teams`)에서 담당을 고르는 함수(`registry.py:57-85`)	그 종류를 받는 에이전트가 정확히 하나여야 합니다. 아니면 `ROUTING_FAILED` → escalated.

`registry.capability_for()`	담당 안에서 일감 종류(capability)를 고르는 함수 (`registry.py:87-128`)	① 에이전트의 `select_capability` 에 묻고 ② 없으면 요청 종류 이름 맞추기 ③ 없으면 `default_capability` . 여행 처리 건 (`subject_ref.kind == "trip"`)이면 `{에이전트}.itinerary` , 고객 신고가 질문이면 `{에이전트}.itinerary_question` .
`team_id`	에이전트 이름. 여행 MVP는 `dining` (요식) · `activity` (액티비티) · `mobility` (이동). `booking_handoff` (예약 넘기기) · `lodging` · `flight` 는 등록만 돼 있고 MVP 밖.	골라지면 `ROUTED` 기록에 `{owner_team_id, capability}` 로 남습니다.
`capability`	일감 종류. **에이전트 이름.하는 일** 모양입니다. 예: `dining.itinerary` = 요식 에이전트의 「여행 일정 관리」. 전체 목록은 8장.	에이전트가 지원하지 않는 종류면 거부(`unsupported_capability`).

---

4. 코어 → 에이전트: 일감 `TeamTask`

무엇인가: 코어가 에이전트를 한 번 부를 때 넘기는 작업 지시서입니다. 정의는 `app/core/contracts.py:165-193` , 만드는 곳은 `Controller._task()` (`app/application/controller.py:131-163`). 모르는 칸이 오면 거부합니다(`extra="forbid"`).

값	무엇인가	어디서 정해지나	어떻게 작용하나	숫자 근거
`contract_name` · `contract_version`	계약 이름 `"a_cop.team_task"` 과 판 `"1.0"`	고정값	에이전트가 같은 계약 · 같은 큰 판(major)일 때만 등록됩니다.	—
`case_id`	**처리 건 번호**. 2장의 처리 건 하나	`customer_cases` 의 번호	어느 처리 건의 일인지. 근거 묶음의 `case_id` 와 같아야 합니다.	—
`case_version`	**처리 건의 판 번호**. 처리 건이 바뀔 때마다 1	일감을 만드는 순간의 `customer_cases.ver`	결과를 반영할 때 판이 그대로인지 봅니다. 그 사이 다른 쓰기가 처리 건을 바꿨으면(`StateConflict`) 결과를 버리고 이번 실행만 실패로 닫습니	—

	씩 오릅니다	`sion`	다. 이유: 「판이 밀렸다는 것은 근거가 더 이상 사실이 아닐 수 있다는 증거」(`controller.py:319-339`).	
`run_id`	**실행 번호**. `run_case` 를 한 번 돌릴 때마다 새로 생기는 실행 기록	`CaseService.start_run()` 이 표 `agent_runs` 에 씀	한 처리 건에 동시에 도는 실행은 1개만 허용(`max_active_runs_per_case: 1`). 끝나면 `succeeded` / `failed` 로 닫습니다.	1 근거 없음
`task_id`	**일감 번호**. 에이전트를 한 번 부를 때마다 새로 만드는 번호(`uuid4()`)	`Controller._task()`	결과(`TeamResult`)가 같은 번호를 돌려줘서 「어느 일감의 결과인지」 짝을 맞춥니다.	—
`team_id`	담당 에이전트 이름(3장)	`registry.resolve`	에이전트가 자기 이름과 다르면 다른 팀으로 넘김(`handoff`)을 돌려줍니다.	—
`capability`	일감 종류(3장, 8장)	`registry.capability_for` (한 번만 고름)	한 번만 고르는 이유: 「두 번 고르면 규정 판단에 쓴 종류와 에이전트가 받는 종류가 갈릴 수 있다」(`controller.py:159-161`).	—
`input_text`	처리 건 제목. 고객 원문, 또는 감시 건이면 `"[감시] {항목 제목} — {요약}"`	`customer_cases.subject`	1~12,000자. 넘으면 일감을 만들 때 오류가 나서 실행이 실패로 닫히고 처리 건은 `running` 에 멈춥니다(14장).	12,000자 근거 없음 — 근거 묶음의 「12,000 토큰」과는 다른 값(글자 수)이고 이어진 기록도 없습니다.
`allowed_tools`	이 일감에서 쓸 수 있는 조회 도구 목록	에이전트 등록 정보(`manifest.allowed_tools`)를 그대로	목록 밖 도구를 부르면 `ToolNotAllowed` 로 거부(7장).	—
`deadline_at`	마감 시각 = 지금 + `reliability.team_timeout_seconds`	`config/guardrails.yaml:46` = 90	코어가 에이전트를 `asyncio.wait_for(..., timeout=90)` 으로 부릅니다. 90초를 넘기면 `GUARDRAIL_ESCALATED` 기록 → 처리 건 `escalated` , 실행 `failed` . **감시 건이면 일정도 안 바뀌고 고객 알림도 없습니다**(오류 기록만). 로컬 에이전트는 `deadline_at` 값을 읽지 않습니다(원격 에이전트만 씀, 꺼져 있음).	90초 근거 없음 (출처로 적힌 「v5 §11」 파일 없음). 참고 사실: 접수→실행 실측 p50 20~34초 · p95 32~51초.

`resume` · `resume_node`	「기다렸다가 이어서 하는 실행인가」와 「어디서부터 이어 하나」. 둘은 짝입니다.	대기 이유에 따라 1:1로 정해짐 (`contracts.py:78-86`)	아래 표 참고. **여행 MVP의 요식 · 액티비티 · 이동 일정 관리는 기다리지 않습니다** — 일정 변경은 승인 없이 바로 적용하거나(`auto_apply`), 고객에게 묻는 것은 처리 건을 기다리게 하지 않고 따로 「보류 제안」 (`pending_changes`)으로 엽니다.	—
`context`	근거 묶음 `ContextPack` (5장)	`ContextBroker.build()`	에이전트가 판단에 쓰는 상태 · 근거를 예산 안에서 묶은 것.	5장

기다렸다가 이어서 하는 경우 — `resume` 이 쓰이는 세 가지

대기 이유 `wait_reason`	이어 할 곳 `resume_node`	무엇을 기다리나	여행 MVP에서
`customer_input`	`validate_input`	에이전트가 「고객에게 이것을 더 받아야 한다」(`wait_for_input` + 받을 모양 `required_input_schema`)를 돌려줬을 때. 처리 건은 `waiting_input` . 고객이 `POST /v1/cases/{case_id}/messages` 로 답하면(재개 표 24시간) 이어서 실행.	**안 쓰임** — 여행 에이전트 중 이것을 돌려주는 곳이 없습니다(코드 검색 0건).
`human_approval`	`execute_approved_action`	에이전트가 「승인이 필요한 실행 제안」(예: 업체 예약 변경 `booking.change`)을 냈을 때. 제안이 `action_requests` 에 `pending_approval` 로 저장되고 처리 건은 `waiting_approval` . 운영자가 `POST /v1/cases/{case_id}/actions/{action_id}/approve` 로 승인하면 코어가 실행.	**일정 관리에선 안 쓰임**. 예약 넘기기(MVP 밖)와 액티비티의 예약 변경 제안 (`activity.propose_change`)에만 있습니다. 승인 뒤 자동으로 이어 돌리는 호출자는 찾지 못했습니다(14장).
`external_callback`	`verify_external_result`	바깥 시스템의 응답을 기다림	**안 쓰임** — 상태표에만 있고 쓰는 코드 0건.

---

5. 근거 묶음 `ContextPack` 과 그것을 만드는 `ContextBroker`

`ContextBroker` 란: 에이전트에게 줄 상태와 근거를 **12,000 토큰 안에** 정해진 규칙대로 잘라 묶는 부품입니다(`app/core/context.py`). 답을 만들지는 않습니다. 코어가 `ContextInputs` (재료)를 주면 `build()` 가 `ContextPack` (묶음)을 돌려줍니다.

`build()` 가 하는 일 (`context.py:199-280`)

1. **자르면 안 되는 것부터 셉니다**: 지시문(`system_instruction` , 지금은 고정 문장 `"Answer using the supplied evidence."`)과 처리 건 상태 (`current_state`). 토큰은 tiktoken `cl100k_base` 로 실제로 셉니다(「글자 수 ÷ 4 같은 어림은 예산 초과를 조용히 통과시킨다」). 지시문 1,800 · 상태 2,400 토큰을 넘으면 자르지 않고 오류를 냅니다.
2. **잘라도 되는 네 칸을 각자 예산에 맞춥니다**: 도구 사실(2,400 · 최신순 · 중복 제거) · 규정 조각(3,600 · 점수 높은 순) · 지난 대화(1,200) · 비슷한 사례(600). 뺀 것은 `omissions` 에 이름을 남깁니다.
3. **규정 조각을 근거로 바꿉니다**: `Evidence(source_type="policy", source_id="{문서}#c{조각}", confidence=점수)` . 전체 근거는 40건까지.
4. **근거가 모자란지 판정**(`degraded`): 규정 검색이 실패했거나, 규정이 필요한 일감인데 0건이면 `degraded=True` . 여행 일정 관리(`*.itinerary`)는 규정이 필요 없는 일감으로 정해져 있어 0건이어도 괜찮습니다.

칸	무엇인가	작용	숫자 근거
`pack_id`	묶음 번호(매번 새로)	—	—
`case_id` · `team_id` · `tenant_id`	처리 건 · 에이전트 · 사용처(테넌트) 번호	일감의 값과 같아야 함	—
`knowledge_scope`	규정 검색 범위. 예: 액티비티 = `travel_activity` · `travel_weather` · `travel_cancellation` · `travel_access`	규정 검색(RAG)을 이 범위 안에서만	—
`current_state`	처리 건 상태(아래 표)	에이전트가 무엇이 · 왜 · 어느 항목인지 알아냄	2,400 토큰 근거 없음

`evidence`	근거 목록(최대 40건)	에이전트가 시작할 때 이미 가진 근거	40 근거 없음 (개수 상한을 두는 이유만 있음: 짧은 근거 100개가 들어오는 것 막기)
`history_summary`	지난 대화 요약(최대 10,000자)	**지금 늘 비어 있음** — 코어가 지난 대화를 넘기지 않음	10,000 근거 없음
`similar_cases`	비슷한 지난 사례(최대 3건)	**지금 늘 비어 있음**	3 근거 없음
`token_budget`	전체 예산 12,000 토큰(바꿀 수 없음)	넘으면 묶음을 만들지 않음	12,000 근거 있음 ① 건당 LLM 비용 통제 ② 긴 입력 중간을 놓치는 품질 저하(lost-in-the-middle) 완화 ③ 실험 재현성. 「모델 한계 때문이 아니다」 (`contracts.py:125-127`). 칸별 나눔(1,800 · 2,400 · 2,400 · 3,600 · 1,200 · 600)은 근거 없음.
`estimated_input_tokens`	실제로 센 토큰 합계	12,000 넘으면 거부	—
`degraded` · `omissions`	근거가 깎였나 · 무엇을 뺐나	깎였으면 에이전트가 `degraded_context` 로 멈추고, 코어도 실행 제안을 자동 적용하지 않음	—

`current_state` 안의 값 (`Controller._task()` , `controller.py:148-152`)

값	무엇인가 · 모양	누가 쓰나	에이전트가 어떻게 쓰나
`case_id` · `customer_id` · `status` · `version`	처리 건 번호 · 고객 번호 · 상태 · 판	처리 건 표	참고용
`intent` · `issue_code` · `sentiment` · `owner_team_id`	요청 종류 · 문제 코드 · 감정 · 담당	분류 · 감시	참고용
`request_id`	요청 번호. 감시 건이면 `watch:{trip}:{item}:{지문}` , 없으면 처리 건 번호	`request_id_for_case()`	중복 방지 키를 만드는 재료(9장)

`subject_ref`	무엇에 대한 건인가. `{kind:"trip", id:여행 번호, version, part_id:항목 번호, part_kind, recent_part_id, base_version, request:{type, at, …}}`	`case_intake.py:84` (감시는 `{"kind":"trip","id":…,"part_id":…}`)	어느 여행 · 어느 항목인지. `kind=="trip"` 이면 일정 관리 일감이 됨
`trigger_source`	누가 시작했나: 고객이면 `"customer"` , 감시면 `"schedule"`	`case_intake.py:76`	`"schedule"` 이면 `handle_trigger()` 로 감
`trigger`	감시가 찾은 것. `{item_id, at:시각, detected:"place"|"route", categories:[원인 종류]}`	감시만 씀	대상 항목과 판단 시각
`interpretation`	접수할 때 고객 문장에서 미리 뽑아 둔 신고. `{routing_hint, report:{type: delay·closed·stock_out·change·rollback·question·other, minutes?, products?}}`	분류 단계 (`case_intake.py:114`)	있으면 같은 문장을 모델에 다시 묻지 않음

---

6. 에이전트 안에서 일어나는 일

`execute(task)` 란

에이전트(팀)마다 하나씩 있는 **「일감을 받아 결과를 돌려주는 함수」**입니다. 부르는 사슬:

1. `Controller.run_case` 가 `asyncio.wait_for(self.team_executor.execute(task), timeout=90)` 으로 부름(`controller.py:299`)
2. `LocalTeamExecutor.execute` (`app/core/remote_team/executor.py:22-31`)가 `registry.get(team_id)` 로 에이전트를 찾고, 일감의 도구 목록이 에이전트 등록 범위 안인지 확인한 뒤(`ToolScopeViolation`) 에이전트의 `execute(task)` 를 부름
3. 에이전트의 `execute` : 먼저 `_guard` 로 검사 → 일감 종류가 `*.itinerary` · `*.itinerary_question` 이면 공통 `run_itinerary(task)` , 아니면 에이전트별 다른 갈래

`_guard` — 시작 전 검사 세 가지 (`_base.py:49-65`)

순서	검사	걸리면
1	일감의 `team_id`가 내 이름이 아님	다른 팀으로 넘김(`handoff`) — 단, 코어는 이 값을 처리하지 못합니다(14장)
2	내가 지원하지 않는 `capability`	`unsupported_capability`로 escalated
3	근거 묶음이 깎임(`degraded`)	`degraded_context`로 escalated

`run_itinerary(task)` — 일정 관리 공통 순서 (`itinerary_team.py:88-135`)

1. `read.itinerary {trip_id: subject_ref.id}` 로 지금 일정 판(여행 + 항목 전체)을 읽음. 없으면 `unknown_여행 일정`
2. 판단 시각 = 요청 시각 또는 `trigger.at` , 둘 다 없으면 지금(한국 시각)
3. `trigger_source == "schedule"` (감시 건)이면 → **`handle_trigger()`** (에이전트별, 8장)
4. 아니면 신고 종류를 정함: 요청 `type` → 없으면 `interpretation.report.type` → 없으면 `read.customer_report`로 문장에서 뽑음. `other` 면 `report_not_understood`
5. 종류별 갈래: `question` → `_answer_question` (규정 답) · `change` → `_change` (다른 곳으로 바꾸기, `plan_swap`) · `rollback` → `_rollback` (옛 판으로) · 그 밖(`delay` 늦음 · `closed` 휴무 · `stock_out` 품절) → **`handle_report()`** (에이전트별)
6. 계산 결과를 `settle()` 이 결과(`TeamResult`)로 바꿈(9장)

조회 도구를 부르는 규칙 — `_read()` → `ReadToolbox.call()` (`read_tools.py:534-563`)

순서	검사	걸리면	숫자 근거
1	일감의 `allowed_tools`에 없는 도구	`ToolNotAllowed`	—
2	등록되지 않은 도구 이름	`ToolNotAllowed`	—

3	같은 도구를 같은 인자로 두 번	`ToolLoopExceeded`	—
4	이번 일감에서 이미 12번 부름(`max_steps=12`)	`ToolBudgetExceeded`	12 부분 근거 액티비티 주석 「대안 후보마다 재점검한다(도구 1회씩) — 6으로는 후보 셋에서 예산이 끝난다」, 위키 「2026-09-17 6 → 12」. 계약이 허용하는 최댓값도 12. 왜 하필 12인지는 없음.

읽은 값은 `_evidence()` 가 근거로 계속 쌓습니다(`_base.py:81-105`): 번호 `tool:{에이전트}:{도구}` , 종류는 도구면 `tool_result` 아니면 `db` , 확신도 1.0 고정(근거 없음). **빈 값은 붙이지 않습니다** — 「빈 근거는 근거가 아니다」.

---

7. 조회 도구 — 무엇을 어디서 읽나 (`app/tools/read_tools.py`)

도구	받는 값	어디서 읽나	돌려주는 것	못 읽으면
`read.itinerary`	`trip_id`	DB `trips` · `itinerary_items` · `places`	여행(`trip_id, version, title, locale, party_size, constraints`)과 항목 전체	`None`
`read.itinerary_version`	`trip_id, version`	같은 표, 판 지정	그 판의 항목들	`None`
`read.place_catalog`	`trip_id`	`places` + `place_catalog` (관광공사 분류)	대안 후보 장소 목록	`None`
`read.customer_report`	`text`	신고 추출기 (`trip_intake.extract` + Ollama 모델)	`{type, minutes?, …}`	`None` 또는 `{type:None, error}`
`read.disruptions`	`place_id, latitude, longitude, weather_sensitive, region="서울", district, starts_at`	외부 소스 점검기 `DisruptionCheck` (날씨 · 특보 · 재난 등을 동시에 조회)	`verdict` : `clear` (문제 없음) · `disrupted` (사건 있음 → 바꿀 대상) · `fatal` (1차 · 대체 소스 모두 실패) + 사건 목록 · 확인 시각	`None`

`read.place`	`place_id`	`places`, 좌표가 비면 국가유산청으로 채움	장소 정보	`None`
`read.policy`	`query`	규정 문서 벡터 검색 (`knowledge_chunks`), 범위 `knowledge_scope`	규정 조각 목록	오류가 그대로 올라감
`read.booking`	`booking_id` (선택)	`bookings`	예약 한 건	`None` — **14장 문제 참고**
`read.booking_terms`	`booking_id`	`bookings` → `supplier_bookings` → `cancellation_terms`	취소 기한 · 위약금율	`None`
`read.dining_state`	`place_id, at, until, order_margin_min`	요식 원장 (`dining.core_place_state()` 등)	그 시각 영업 여부 · 라스트오더 가능 · 브레이크 · 식이 조건 · 확인 시각	`None` 또는 `available=False`
`read.dining_alternatives`	`place_id, at, until, conds, next_lat, next_lng`	요식 원장 `suggest_alternatives()`	대신 갈 식당 후보(축: 덜 밀리는 곳 · 비슷한 곳 · 가까운 곳)	`available=False`
`read.route_events`	`targets` (구간)	UTIC 교통 사건(도로만 답함)	`{events, unsupported}`	`None` 또는 `events=None`
`read.route` · `read.transit`	—	**미구현**	늘 `None`	—

에이전트별 허용 도구: 공통 4개(`ITINERARY_TOOLS`) = `read.itinerary` · `read.itinerary_version` · `read.place_catalog` · `read.customer_report`. 요식 = 공통 + `read.place` · `read.policy` · `read.booking` · `read.booking_terms` · `read.dining_state` · `read.dining_alternatives` · `read.disruptions`. 액티비티 = 공통 + `read.booking` · `read.booking_terms` · `read.policy` · `read.place` · `read.disruptions`. 이동 = 공통 + `read.route` · `read.transit` · `read.policy` · `read.route_events`.

8. 에이전트별 — 받는 것 · 하는 일 · 돌려주는 것

요식 `dining` (`app/modules/travel_ops/dining/team.py`)

일감 종류	하는 일	언제 쓰이나	돌려주는 값
`dining.itinerary`	여행 일정의 식사 관리 — 감시 · 늦음 · 휴무 · 다시 바꿔 줘	여행 처리 건(규정 검색 면제)	9장 `settle()` 결과
`dining.itinerary_question`	규정 질문 답	여행 처리 건 + 신고가 질문	답 문장, `decisions=[{"itinerary":"question_answered", …}]` , 확신도 0.7
`dining.check_open` (기본)	예약 시각에 여는가	여행 처리 건이 아닐 때	`decisions=[{"open_at_slot", "confirmed_at", …}]` , 「일정 시각에 영업합니다」 류 답, 확신도 0.9
`dining.check_conditions`	식이 조건(할랄 · 채식 등) 대조	**고르는 경로가 코드에 없음** (14장)	`{"conditions_met", "requested"}`

감시 건 `handle_trigger()` : ① `trigger.item_id` 항목 찾기(없으면 「이미 바뀜」 `gone`) ② `read.disruptions` → `fatal` 이면 `fatal_source_failure` , `disrupted` 가 아니면 「영향 없음」 `clear` ③ 후보 장소를 읽고 `plan_dining_disrupted()` 가 대신 갈 식당을 고름 — 요식 원장 후보 먼저, 다 떨어지면 장소 목록. 후보는 최대 3곳까지 다시 점검(`DINING_RECHECK_LIMIT=3` , 근거 있음 「도구 예산 12 안에서 쓰고 남는 몫」), 점검 결과가 `clear` 가 아니면 탈락.

고객 신고 `handle_report()` : `delay` (늦음) → `plan_delay()` , `closed` (휴무) → `plan_closed()` . 반경 700m(근거 「우리가 고른 값 — 도보 약 9분」), 라스트오더 탈락 기준 도착 + 20분(근거 없음), 좌석 여유 10분(근거 없음).

액티비티 `activity` (`activity/team.py`)

일감 종류	하는 일	돌려주는 값

`activity.itinerary`	여행 일정의 활동 관리 — 감시 · 품절 · 다시 바꿔 줘	9장 `settle()` 결과, 또는 「바꿀까요?」 제안
`activity.itinerary_question`	규정 질문 답	요식과 같음
`activity.check_feasible` (기본)	그 시각에 성립하나 — 이미 시작 · 인원 초과 · 점검 결과	`{"feasible", "reason"…}`, 확신도 1.0/0.8. 사건이 있으면 아래 예약 변경 제안으로
`activity.check_cancelable`	지금 취소 가능 · 위약금	`{"cancelable", "hours_remaining", "penalty_rate"…}` — **고르는 경로가 코드에 없음**
`activity.propose_change`	업체 예약 변경 제안(`booking.change`), **승인 대기**	확신도 0.6, `wait_for_approval`

감시 건: `read.disruptions` 까지는 요식과 같고, 이어서 ① 실내인지 모르거나 설문에서 자동 변경을 고르지 않았으면 **「바꿀까요?」만 묻는 제안** (`consent_arguments()` , 대체안 없음) ② 아니면 `plan_activity_adjustment()` 가 같은 대분류 안에서 대체 활동을 고름(반경 600m, 근거 「같은 건물 · 근처 실내로 피한다」). **고객 신고**: `stock_out` (품절)만 — 일정은 안 바꾸고 근처 가게를 답으로.

이동 `mobility` (`mobility/team.py`)

일감 종류	하는 일	돌려주는 값
`mobility.itinerary`	감시 건 — 구간 사건(통제 · 사고)이 계획 경로에 걸리면 경로를 다시 고름	9장 `settle()` 결과
`mobility.itinerary_question`	규정 질문 답	요식과 같음
`mobility.check_route` (기본)	이동이 일정 안에 들어가나	조회 갈래 `{"fits","need_minutes","gap_minutes","last_departure_ok"}` — **도구 미구현이라 실제로는 늘 「모름」**. 시간표 판정기 갈래(`current_state.mobility` 가 올 때)는 구간마다 `{"leg_id","verdict": ok|fail|unknown,"evidence_grade"}` , 확신도 확정 0.95 · 추정 0.6 · 근거 없음 0.2
`mobility.status` · `mobility.exception`	운행 상태 · 놓침 · 결항 문장	고르는 경로가 코드에 없거나 미구현

감시 건: `read.route_events` 로 구간 사건을 읽고(못 읽으면 `fatal_source_failure`) `plan_route_adjustment()` 가 막힌 경로 대신 다른 선택지를 고름, 다 막히면 시간표 판정기로 다시 찾고, 그래도 없으면 「해결 못 함」(`unresolved`). **고객 신고**: 다루지 않음.

「앞뒤 이동 다시 계산」은 이동 에이전트가 하는 일이 아닙니다. 요식 · 액티비티가 장소를 바꾸면, 코어가 변경을 적용할 때 `refresh_moves_around()` 가 앞뒤 이동 항목을 다시 만듭니다(10장).

---

9. 에이전트 → 코어: 결과 `TeamResult`

무엇인가: 에이전트가 일감 하나를 마치고 돌려주는 결과지입니다(`contracts.py:223-242`). 공통 함수 `settle()` (`itinerary_team.py:254-280`)이 계산 결과를 이 모양으로 바꿉니다.

값	무엇인가	코어가 실제로 쓰나
`task_id` · `run_id` · `team_id`	어느 일감 · 실행 · 에이전트의 결과인가	사실 대조 실패 기록에만
`outcome`	결과 종류: `completed` (마침) · `waiting` (기다림) · `handoff` (다른 팀으로) · `escalated` (못 함) · `failed` (실패)	**안 읽음** — 코어는 아래 `next_action` 만 봄
`next_action`	다음 할 일 — 아래 표	**갈래를 가르는 유일한 값**
`answer`	고객용 문장(최대 6,000자). 바꿨으면 알림 문구 `plan.notice["text"]`	처리 건 기록(`state_patch.answer`)에 저장. 고객에게 바로 가지 않음 — 알림은 10장 적용기가 따로 만듦. 6,000 근거 없음
`confidence`	확신도 0~1. 값: 1.0(확정 판정) · 0.9(일정 변경 · 그대로) · 0.8 · 0.7(규정 답) · 0.6 · 0.0(못 함)	안 읽음. 값들 근거 없음
`evidence`	근거 목록. 칸: `evidence_id` (이름표) · `source_type` (`customer_message` 고객 말 · `db` · `policy` 규정 · `tool_result` 도구 · `case_event` · `remote_agent` 남의 시스템이 말한 것) · `source_id` (어디서) · `claim` (무엇을 확인) · `value` (값) · `confidence` · `observed_at` (확인 시각)	기록에 저장. 사실 대조에는 **코어가 만든 근거 묶음의 근거만** 씀 — 에이전트가 근거와 제안을 함께 지어내면 대조가 순환하므로

`decisions`	판단 기록(자유 모양). 예: `{"itinerary": 이유, …요약}`	안 읽음
`action_proposals`	실행 제안 목록 — 아래	읽음(10장)
`wait_reason` · `required_input_schema` · `handoff_capability`	기다리는 이유 · 고객에게 받을 모양 · 넘길 일감 종류	4장 기다림 경우에만
`failure_code` · `warnings`	못 한 이유. 예: `fatal_source_failure` (소스가 대체까지 실패) · `unknown_{무엇}` (확인 못 함 — 값이 없는 게 아니라 모름) · `itinerary_unresolved` (대안 없음) · `target_kind_mismatch`	escalated 기록의 `guardrail` · `observed` 가 됨

`next_action` — 다음 할 일 (`contracts.py:68-75`)

값	뜻	코어 처리	처리 건 상태
`respond`	답하고 끝(승인 없는 제안이 붙어 있으면 적용)	`COMPLETED`	`resolved`
`wait_for_input`	고객 입력 기다리기	`MISSING_INPUT`	`waiting_input`
`wait_for_approval`	승인 기다리기	`APPROVAL_REQUIRED`	`waiting_approval`
`escalate`	못 함	`GUARDRAIL_ESCALATED`	`escalated` — 오류 기록. **감시 건이면 고객 알림 없음**
`continue` · `call_tool` · `handoff`	계속 · 도구 호출 · 넘기기	**처리 갈래 없음** → 오류	`running` 에 멈춤(14장)

그대로 둘 때의 답 — `ANSWERS` (`itinerary_team.py:46-52`)

상태	고객 문장
`still_fits`	지금 일정 그대로도 괜찮아요 — 바꾸지 않았어요.

`clear`	지금 확인해 보니 일정에 영향이 없어요 — 바꾸지 않았어요.
`gone`	그 일정은 이미 바뀌었어요 — 다시 바꾸지 않았어요.
`no_meal`	그 시각 뒤에는 식사 일정이 없어요.
`no_alternate`	바꿀 수 있는 다른 안이 없어요.

이 다섯 밖의 「못 바꿈」(`unresolved` · `stale` · `not_today` 등)은 `itinerary_{상태}` 로 escalated가 됩니다.

실행 제안 `ActionProposal` (`contracts.py:201-215`)

값	무엇인가	여행 일정 변경일 때
`action_type`	무슨 실행인가	`"itinerary.apply"` (일정 새 판 적용)
`arguments`	실행에 필요한 값	`change_arguments()` : `trip_id` · `base_version` (어느 판을 기준으로 바꿨나) · `reason` (`auto_adjusted` 감시 · `customer_report` 신고 · `customer_request` 요청 · `rollback`) · `causes` (원인) · `notice` (알림 문구 · 다른 안) · `summary` + `replacements:[{item_id, item}]` (바뀐 항목만) 또는 `full_items` (되돌릴 때 통째로). 「바꿀까요?」만 물을 땐 `consent_arguments()` : `reason:"indoor_unknown"` , `consent:{item_id, indoor_unknown}`
`idempotency_key`	중복 방지 키 (8~128자)	에이전트 값은 참고용. **서버가 다시 계산한 값이 최종**: `sha256( sha256(tenant_id) + sha256(request_id) + sha256(action_type) + sha256(대상) )` , 대상 = `"{trip_id}:v{base_version}"` — 같은 여행 · 같은 기준 판은 한 번만 적용. 서버가 계산하는 이유: 처리 건 번호로 키를 만들던 때 한 처리 건의 둘째 제안이 조용히 사라진 일(2026-09-10 실측, v11 §4-E)
`approval_required`	승인이 필요한가	`False` — 우리 DB 안의 일정 판만 바꾸므로 승인 없이 먼저 고치고 알림(v11 §4-C)
`risk_level`	위험도 `low` / `medium` / `high`	`"low"` — 승인 없이 적용되려면 반드시 low

`rationale_evidence_ids`	제안의 근거 번호	비어 있음(`[]`)

결과 양식이 스스로 막는 것 (`contracts.py:244-295`)

- 답(`answer`)이 있는데 근거(`evidence`)가 없으면 거부 — 「근거 없는 답변 금지」
- 승인이 필요한 제안이 있으면 다음 할 일은 반드시 `wait_for_approval` — 「에이전트는 승인 없이 실행하지 않는다」
- 제안의 근거 번호는 같은 결과의 근거 안에 있어야 함
- `respond` 면 답 필수 · `escalate` 면 이유(`failure_code` 나 `warnings`) 필수 · `wait_for_input` 이면 받을 모양 필수 · `handoff` 면 넘길 일감 필수

---

10. 코어가 결과를 반영 — `Controller._apply_result()` → `_event_for_result()`

결과 반영은 처리 건 상태 바꾸기 · 기록 · 알림 넣기를 **한 묶음(트랜잭션)**으로 합니다. 유일한 문은 `transition_case()` (`app/core/transition.py`)이고, 판 번호가 맞을 때만 씁니다.

`respond` + 승인 없는 제안 — 감시 건이 일정을 바꾸는 길

1. `_reject_unverified()` — 제안을 사실과 대조. 근거 묶음이 깎였으면 막음(`degraded_context_blocks_action`). 제안의 값을 **고객 번호 조건으로 DB에서 다시 읽어** 대조: `trip_id → trips` , `booking_id → bookings` , 인원 ≤ 정원 등. 일정 칸(`base_version` · `replacements` 등)은 여기서 안 보고 적용기가 적용하는 순간 봄. 어긋나면 `action_proposal_verification_failed` .
2. `_apply_auto_proposals()` — 제안 전부를 한 묶음(savepoint)으로 적용, 하나라도 실패하면 전부 되돌림.
   - 적용기 찾기: `action_handlers["itinerary.apply"]` = **`ItineraryApply`** (`auto_apply=True`). 없으면 `action_handler_missing`
   - `risk_level` 이 `low` 가 아니면 `action_requires_approval`
   - 중복 방지 키를 서버가 다시 계산 → 이미 성공한 키면 다시 적용하지 않음(`replayed`)
   - **`ItineraryApply.apply()`** 실행(아래)

3. 성공하면 실행 기록(`action_requests` , `succeeded`) · 처리 건 `COMPLETED` → `resolved` · 알림 메시지를 같은 묶음에 넣음

`ItineraryApply.apply()` 단계 (`itinerary_actions.py:74-160`)

1. `TripStore.latest()` 로 최신 판을 읽음. 없거나 **다른 고객의 여행이면 같은 문구로 거부**(있는지도 말하지 않음)
2. 지금 판이 `base_version` 과 다르면 충돌(`ActionConflict` → `action_target_changed`)
3. 「바꿀까요?」 제안(`consent`)이면 `_ask_consent()` — 일정은 안 바꾸고 보류 제안을 열고 묻는 알림만 넣음
4. 새 항목 목록 만들기: `full_items` 면 그대로, 아니면 `replacements` 로 해당 항목만 갈아 끼움(일정에 없는 항목이면 거부)
5. **`refresh_moves_around()`** — 장소가 바뀐 항목의 바로 앞뒤 이동 항목을 다시 만듦. 시간표 판정기가 있으면 다시 판정, 없고 걸어갈 거리(1,200m, 근거 「초속 1.04m 기준 약 19분」) 안이면 이름만 바꿈, 멀면 직선 어림값
6. 새 장소가 이 여행의 장소 목록에 없으면 거부(지어낸 장소 금지)
7. **설문 확인 — `_ask_instead()`** : 고객이 직접 고른 변경(`customer_request` · `rollback` · `customer_choice`)이 아니면 `pending.decide()` 로 판정. 위에서부터 먼저 걸리는 것이 이김: ① 바꾸지 않기로 한 일정(고객 고정 · 잠김 · 예약번호 있음) → 묻기 ② 설문 「먼저 물어봐」 + 지진 · 재난문자 · 기상 「경보」 → 안전 알림 ③ 설문 「먼저 물어봐」 → 묻기 ④ 그 밖 → 적용. 묻기면 일정은 안 바꾸고 보류 제안(`pending_changes` , 그 일정이 끝나면 만료 · 답 없으면 그대로) 과 묻는 알림만 넣음
8. **`store.append_version()`** — 새 일정 판 저장. `trips.latest_version` 을 판 번호 조건으로 올리고 `itinerary_versions` · `itinerary_items` 에 새 판을 씀. **옛 판은 지우지 않음**
9. 알림 메시지 1건: 주제 `trip.notice` , 중복 방지 `"{trip_id}:v{판}"` , 내용 = 알림 문구 · 여행계획서 링크 · 판 번호(감시가 바꾼 것이면 「되돌리기」 제안 포함)

그 밖의 갈래

- `wait_for_approval` : 사실 대조 → 제안마다 `action_requests(status="pending_approval")` → `waiting_approval` . 승인 뒤 코어가 직접 실행(`_apply_approved`), 바깥 응답이 끊기면 `unknown` 으로 두고 다시 실행하지 않음
- `wait_for_input` : 재개 표 발급 → `waiting_input`

- `escalate` : `GUARDRAIL_ESCALATED` → `escalated` . **알림 메시지가 붙지 않습니다** — 감시 건이면 고객은 아무것도 받지 않고, 여행계획서 링크는 바뀌지 않은 일정을 보여 줍니다. 운영 화면에 빨간색으로 표시

---

11. 알림 — 보낼 함 → 배달 일꾼 → 디스코드

단계	이름	하는 일	숫자 근거
보낼 함	`OutboxMessage(topic, payload, dedupe_key)` → 표 `outbox`	처리 건 상태 변경과 **같은 묶음**으로 넣음. `UNIQUE(tenant_id, topic, dedupe_key)` 라 같은 알림은 한 번만	—
배달 일꾼	`OutboxWorker` (`app/infrastructure/messaging/worker.py`), 실행 `scripts/run_outbox_worker.py`	대기 중인 알림을 한 건씩 꺼내 보냄. 성공 → `delivered` , 시간 초과 · 연결 끊김 → `unknown` (갔는지 모르니 다시 보내지 않음 — 두 번 가는 것도 사고), 그 밖 실패 → 1분 뒤 재시도, 3번째 실패면 `dead_letter`	3번 · 1분 근거 없음 . 처리 중 멈춘 알림 회수 300초 근거 없음
디스코드	`app/infrastructure/notify/discord.py` , 설정 `ACOP_DISCORD_WEBHOOK_URL`	**모든 고객의 알림이 웹훅 하나(팀 시연 채널)로 갑니다.** 고객별 웹훅은 저장만 되고 발송은 연결 전(2026-10-01 「사용자 확인 뒤」로 미룸). 설정이 비면 실패로 처리 (보내지 않은 알림을 보낸 것으로 찍지 않으려고)	본문 2,000자 근거 디스코드 한도. 8초 시간 제한 근거 없음

---

12. 고객 채팅 경로 — 코어 · 에이전트를 거치지 않음

`POST /v1/trips/{trip_id}/messages` (`trip_messages.py`)는 처리 건을 만들고 분류는 하지만, 처리는 **`TripDesk` 가 직접** 하고 처리 건을 `transition_case(actor_type="api")` 로 닫습니다. 그래서 4장의 90초 제한, 9장의 결과 양식이 적용되지 않습니다.

경우	고객 문장	처리 건 상태
바꿈 · 답함 · 그대로 · 물음 · 되돌림 등	각 상황 문장(`ANSWERS` · `OUTCOME_ANSWERS`)	`resolved`

분류 실패	「말씀을 분류하지 못해 처리하지 못했어요.」 + 할 수 있는 일	`escalated` (오류 기록)
규정 검색 없음 · 오류	「…답하지 못했어요. 잠시 뒤 다시 물어봐 주세요. 일정은 바꾸지 않았어요.」	`escalated`
처리할 수 없는 종류	「… 이 요청은 지금 처리할 수 없는 종류라 일정은 바꾸지 않았어요」	`escalated`

고객 문장 안에는 「사람」 · 「담당자」 표현이 없습니다(2026-09-28 결정 반영 확인, `trip_replies.py:151-154`).

---

13. 숫자와 근거 한눈에

숫자	어디	근거
12,000 토큰	근거 묶음 예산	있음 비용 · 긴 입력 품질 저하 · 재현성
600초(10분)	멈춘 처리 건 되잡기 기준	있음 정상 실행(수십 초)을 되잡지 않으려고
1분	감시 · 되잡기 주기	있음 주기는 복구 지연을 정한다
3분	외부 6종 소스별 수집(D-017)	있음 감지 → 알림 5분 목표에서 역산 · 코드에 없음
12번	도구 호출 한도	부분 6으로는 후보 셋에서 끝남 · 계약 최대값
3곳	요식 대안 재점검	있음 도구 예산 12 안에서 남는 몫
700m · 600m · 1,200m	요식 반경 · 활동 반경 · 도보 상한	있음 도보 약 9분 · 근처 실내 · 초속 1.04m 약 19분
2,000자	디스코드 본문	있음 디스코드 한도
12,000자	일감의 고객 문장	없음

90초 · 180초	에이전트 시간 제한 · 처리 건 전체 시간	없음 (「v5 §11」 파일 없음)
40 · 10,000자 · 3	근거 수 · 대화 요약 · 비슷한 사례	없음
1,800 · 2,400 · 2,400 · 3,600 · 1,200 · 600	근거 묶음 칸별 예산	없음
6,000자	결과의 답 문장	없음
3번 · 1분 · 300초 · 8초	알림 재시도 · 간격 · 회수 · 시간 제한	없음
24시간 · 1 · 50 · 90분	재개 표 · 동시 실행 · 되잡기 건수 · 감시가 앞을 보는 시간	없음
20분 · 10분	라스트오더 탈락 · 좌석 여유	없음
1.0 · 0.9 · 0.8 · 0.7 · 0.6 · 0.0	확신도 값	없음

근거를 찾은 범위: 코드 주석(`app/**` · `scripts/**`), `config/guardrails.yaml` · `project.yaml` , `final_project_cs/wiki/**` , 루트 `wiki/**` , 구현계획서 v9 · v11. 「구현계획서 v5」는 워크스페이스에 없어 확인하지 못했습니다.

---

14. 코드에서 확인된 문제

#	문제	영향	위치
1	**외부 6종 3분 수집이 구현되지 않음** (D-017 초안 · 「아직 코드에 없다」)	발표 자료의 「3분마다」는 설계. 지금은 1분 · 90분 앞 항목별 점검	`wiki/decisions/D-017…md:99`
2	감시 건이 `escalated` 가 되면 고객 알림이 없음	「사람 대기 없음」은 지켜지지만, 고객은 문제가 있었는지도 모름	`controller.py:352-354`

3	코어가 `handoff` · `continue` · `call_tool`을 처리하지 않음	그 결과가 오면 처리 건이 `running`에 멈추고 되잡히지 않음. `_guard`의 「다른 팀으로」도 여기에 해당	`controller.py:507`
4	12,000자 넘는 문장, 근거 묶음 예산 초과 오류	실행 실패 후 처리 건이 `running`에 멈춤(되잡기는 `routing`만 봄)	`controller.py:340-344`
5	`read.route` · `read.transit` 미구현	이동의 조회 갈래는 늘 「모름」. 시간표 판정기 입력(`current_state.mobility`)도 코어가 넘기는 값 목록에 없음(시험에서만 채움)	`read_tools.py:507-513` · `controller.py:87`
6	`read.booking`이 `case_id` 인자를 버림	그 처리 건의 예약이 아니라 그 고객의 가장 가까운 예약을 읽음	`read_tools.py:169-188`
7	도구 한도 초과를 요식만 잡음	액티비티는 한도 초과 오류가 밖으로 빠져 실행 실패	`itinerary_team.py:230-238`
8	고를 길이 없는 일감 종류	`dining.check_conditions` · `activity.check_cancelable` · `mobility.status` · `mobility.exception`	`registry.py:124-127`
9	감시 건이 「묻기」로 바뀌어도 처리 건 기록의 답은 「...로 바꿨습니다」	고객 알림은 묻는 문구라 맞지만, 기록과 실제가 어긋남	`controller.py:438`
10	설문을 건너뛴 여행의 판정이 경로마다 다름	일반 경로(`decide`)는 건너뛰면 바로 적용, 액티비티 날씨 경로(`auto_on_disruption`)는 건너뛰면 묻기	`pending.py:70-118`
11	승인 뒤 이어서 실행할 자동 호출자 없음	승인 API는 상태만 바꾸고 `run_case`를 부르지 않음(MVP 일정 관리와는 무관)	`cases.py:196-264`
12	「사람」 문구가 남은 곳	예약 넘기기 답 「사람이 확인해야 합니다」(MVP 밖), 운영 화면 「벗어난 건은 그대로 사람에게 옵니다」 등 9곳	`booking_handoff.py:89-90` · `ui/routes.py:349, 424, 698, 776, 789, 854, 875`

코드를 고치는 것은 cs 담당 세션의 몫이라 이 문서는 기록만 합니다.

15. 제가 앞서 드린 설명 중 바로잡은 것

- 「90초를 넘으면 사람에게 넘어갑니다」 → **오류로 기록(`escalated`)되고, 감시 건이면 일정은 그대로 · 고객 알림 없음**. 사람 대기로 넘기는 경로는 고객 쪽에 없습니다.
- 「요식이 식당을 바꾸면 코어가 이동 에이전트에게 다시 일감을 준다」 → **틀림**. 적용기 안의 `refresh_moves_around()` 가 앞뒤 이동을 다시 만듭니다.
- 「고객 요청도 코어 → 에이전트 경로를 탄다」 → **틀림**. 고객 채팅은 `TripDesk` 가 직접 처리합니다(12장).
- 「외부 6종을 3분마다 수집」 → **결정만 있고 미구현**. 지금은 1분 주기 · 90분 앞 항목 점검.
- 「입구 셋 중 고객 API가 처리 건을 열어 에이전트에게 간다」 → 고객 여행 API는 에이전트를 거치지 않습니다(2장 표).

조사: 코드 읽기 세 갈래(일감 · 에이전트 안 · 결과와 적용). 확인 범위 밖: 실제 서버에서 각 경로를 돌려 보지는 않았습니다(코드 읽기만). 「구현계획서 v5」 원문은 없어 그 문서를 출처로 든 숫자의 원 근거는 확인하지 못했습니다.